

Expression Language



Introducción a Expression Language

Expression Language (EL) es una sintaxis incorporada en JSP que **permite acceder y mostrar datos de forma simple dentro de una página Web dinámica.**

Su principal objetivo es que **evita escribir código Java directamente en el JSP**, mejorando la claridad, legibilidad y mantenimiento de la vista.

Antes de EL, muchas tareas simples requerían scriptlets con código Java incrustado. Con EL, esas operaciones pueden resolverse de forma más limpia y expresiva.

Su sintaxis es: **`${expresión}`**

Introducción a Expression Language (Cont.)

¿Para qué se utiliza?

- ▶ acceder a atributos enviados desde servlets o beans
- ▶ mostrar datos en pantalla
- ▶ evaluar condiciones simples
- ▶ trabajar con colecciones y scopes

Por ejemplo, `$\{\text{usuario.nombre}\}$` obtiene la propiedad “nombre” del objeto “usuario”.

Características de Expression Language

- ▶ **Sintaxis simplificada:** EL utiliza una sintaxis simple para acceder a propiedades de objetos, datos de colecciones como listas y maps, resultados de métodos, y variables implícitas sin necesidad de escribir código Java.
- ▶ **Acceso a datos de beans:** EL permite acceder fácilmente a las propiedades de los JavaBeans utilizando la notación de punto o corchetes. Por ejemplo, `${persona.nombre}` o `${persona['nombre']}` para acceder al nombre de una persona siendo “persona” es un bean con una propiedad “nombre”.

Características de Expression Language (Cont.)

- ▶ **Soporte para colecciones:** EL proporciona soporte directo para acceder a datos dentro de colecciones (listas, maps, arrays, etc.). Por ejemplo, `${personas[0].nombre}` para acceder al nombre de la primera persona en una lista de personas.
- ▶ **Operadores:** EL incluye operadores para realizar operaciones aritméticas, lógicas, y relacionales directamente dentro de las expresiones, como `${suma + 1}`, `${persona.edad >= 18}`, y `${not empty list}`.
- ▶ **Coalescencia de Nulos (Null Coalescing):** EL permite operaciones que manejan valores null de manera segura, evitando `NullPointerException` que a menudo ocurre en Java.

Características de Expression Language (Cont.)

- ▶ **Funciones:** EL permite el uso de funciones definidas, lo que puede ser muy útil para manipular datos o realizar cálculos específicos. Las funciones deben ser definidas en una biblioteca de etiquetas o especificadas en el descriptor de despliegue.

Tipos de Expresiones

- ▶ **Expresiones literales:** Incluyen textos, números, booleanos, y null. Por ejemplo, 'Hola Mundo', 123, true, null.
- ▶ **Expresiones aritméticas:** Usadas para realizar operaciones matemáticas. Por ejemplo, `${cantidad + 10}`, `${sueldo * 0.15}`, `${cantidad - 200}`, `${cantidad / total}`.
- ▶ **Expresiones lógicas:** Involucran operadores lógicos y se usan para evaluar condiciones. Por ejemplo, `${edad >= 18}`, `${sueldo > 50000 and tieneBeneficios}`, `${numero == 1}`, `${usuario.rol != 'admin'}`.
- ▶ **Expresiones de método:** Permiten invocar métodos de beans. Por ejemplo, `${bean.calcular()}`, `${persona.getNombre()}`.

Tipos de Expresiones (Cont.)

- ▶ **Expresiones de acceso a propiedades y colecciones:**
Permiten acceder a propiedades de objetos o elementos de colecciones. Por ejemplo, `${persona.nombre}`, `${personas[0]}`, `${config['maxSize']}`

Operadores de EL

- ▶ **and y or:** Son operadores lógicos que corresponden a `&&` y `||` en Java, respectivamente.
- ▶ **eq y ne:** Son operadores de igualdad que corresponden a `==` y `!=` en Java. Son especialmente útiles para comparar cadenas de texto dado que en Java la comparación directa de strings con `==` verifica si dos referencias apuntan al mismo objeto, no si tienen el mismo contenido. Lo mismo con `!=`.
- ▶ **lt, le, gt, y ge:** Son operadores de comparación que corresponden a `<`, `<=`, `>`, y `>=` en Java.

Identificadores reservados en EL

- ▶ **true, false:** Valores booleanos.
- ▶ **null:** Representa un valor nulo.
- ▶ **instanceof:** Utilizado para comprobar el tipo de un objeto.
- ▶ **empty:** Operador que verifica si una cadena está vacía, una colección está vacía, o un valor es null.

Por ejemplo, `${usuario.rol eq 'admin' and usuario.estado ne 'inactivo'}` verifica que el usuario tenga rol “admin” (*eq*) y que su estado sea distinto de “inactivo” (*ne*). El operador *and* indica que ambas condiciones deben cumplirse al mismo tiempo.

Objetos implícitos

Estos objetos **proporcionan acceso a diferentes alcances y contextos de la aplicación**, facilitando la gestión de atributos y la interacción con el entorno del servlet:

- ▶ **pageScope:** contiene datos disponibles únicamente en la página actual.
- ▶ **requestScope:** incluye atributos asociados a la solicitud HTTP en curso.
- ▶ **sessionScope:** permite acceder a datos que se mantienen durante la sesión del usuario.
- ▶ **applicationScope:** contiene información compartida por toda la aplicación.

Objetos implícitos (Cont.)

Continuando con lo anterior:

- ▶ **param:** permite obtener parámetros enviados en el request.
- ▶ **paramValues:** permite acceder a múltiples valores para un mismo parámetro (por ejemplo, checkboxes).
- ▶ **header:** permite acceder a los encabezados de la solicitud HTTP.
- ▶ **headerValues:** útil cuando un encabezado tiene más de un valor.
- ▶ **cookie:** permite acceder a las cookies enviadas por el cliente.
- ▶ **pageContext:** proporciona acceso a información general del entorno, incluyendo request, session y otros objetos del servidor.

Funciones EL

Implementar funciones en Expression Language (EL) propias para usar en JSP es un proceso que involucra varios pasos. Esto **permite extender la funcionalidad de EL con lógica específica que puede ser reutilizada** a lo largo de nuestras páginas JSP.

► Paso 1: Crear las funciones en Java

Se desarrollan las funciones que se desean utilizar dentro de los archivos JSP como **métodos estáticos** en una clase Java.

```
public class MyFunctions {  
    public static String hello(String name) {  
        return "¡Hola " + name + "!";  
    }  
}
```

Funciones EL (Cont.)

- ▶ **Paso 2: Crear un archivo de descriptor de la biblioteca de etiquetas (TLD)**

Luego, se debe definir estas funciones en un archivo Tag Library Descriptor (TLD). Este archivo XML describe las funciones y las etiquetas que están disponibles para ser utilizadas en los archivos JSP.

- ▶ **Paso 3: Registrar el archivo TLD**

El archivo TLD se debe incorporar en el proyecto Web dentro de un directorio como puede ser WEB-INF o en algún otro lugar dentro del classpath de la aplicación.

- ▶ **Paso 4: Declarar la biblioteca de etiquetas en un archivo JSP**

Para usar la función dentro de un JSP, se debe declarar la biblioteca de etiquetas con la directiva **taglib**.

